



Engineering Clinics-II

ECS3001

Title:- Embedded Systems for Continuous Health Monitoring and Early Warning

Semester:- Winter Semester 2025-2026

Guide: Ramkumar D

Team Members

1. GANDRA SAI NIHAR -23BCE9715
2. MENTA SUPRATHIK-23BEC7103
3. GUMMARAJU SAI KOUSHIK-23BEC7151
4. MOHANISH GUNDA-23BEC7157
5. BOLLIPALLI MAHITH SAI-23BEC7277
6. GANJI POOJITHA LAKSHMI-23BEC7354

INDEX

S.No.	Topic	Page No.
1	Abstract	3
	Introduction	4-5
	Background	6-7
	Problem Definition	8-9
4	Objectives	10-11
5	Methodology	12-22
	Required Components — Hardware & Software	
	Working of Required Components	
	Hardware Setup	
	Software Implementation	
	Testing and Deployment	
6	Block Diagram	23-24
7	Plan of Action	24
8	Results & Discussion	25-27
9	Conclusion & Future Scope	28
10	Code Links & Reference Images	29-32

ABSTRACT

The exponential rise in chronic diseases, aging populations, and the increasing demand for real-time healthcare monitoring has necessitated the development of smart, embedded health monitoring systems. This project presents MediPulse, an end-to-end Embedded Systems-based Remote Patient Monitoring (RPM) system designed for continuous health monitoring and early warning generation. The system seamlessly integrates hardware-level sensing with cloud-grade software infrastructure to provide an intelligent, real-time clinical dashboard.

At the hardware core, an ESP8266 Wi-Fi microcontroller interfaces with a MAX30102 pulse oximetry sensor to measure heart rate (HR) and blood oxygen saturation (SpO2), and an MPU6050 six-axis Inertial Measurement Unit (IMU) for activity classification and fall detection. A 16x2 I2C LCD provides on-device visual feedback. Sensor readings are smoothed using a moving average algorithm and transmitted via HTTP POST requests to a backend server running on Node.js and Express.js. Data is persisted in a MongoDB database, and real-time updates are pushed to the dashboard via Socket.IO WebSocket connections.

The software stack features a sophisticated web-based clinical dashboard built with vanilla JavaScript, Chart.js for live waveform visualisation, and jsPDF with html2canvas for downloadable PDF medical reports. The dashboard includes a full authentication system (sign-in and sign-up), patient registration, risk score computation using a multi-parameter algorithm, automated alert generation for abnormal vitals, and an automated email reporting module using Nodemailer to notify guardians and caregivers.

The system is designed with graceful degradation: if the MAX30102 sensor is unavailable, the system continues transmitting accelerometer data without halting. This resilience ensures continuous data collection even under partial hardware failure. Experimental results demonstrate sub-second end-to-end data propagation from the ESP8266 to the dashboard, accurate SpO2 readings within $\pm 2\%$, heart rate within ± 3 BPM, and reliable fall detection at acceleration magnitudes exceeding 3g.

MediPulse addresses a critical gap in affordable, real-time remote patient monitoring by combining low-cost IoT hardware with production-grade software practices. Future enhancements include AI/ML-based anomaly prediction, integration with hospital EMR systems, wearable miniaturisation, and multi-patient multi-ward support.

1. INTRODUCTION

Healthcare delivery is undergoing a fundamental transformation driven by advances in microelectronics, wireless communication, and cloud computing. Traditional hospital-centric models of patient monitoring, which rely on bedside equipment and periodic manual checks by nursing staff, are increasingly insufficient in the face of surging patient numbers, resource constraints, and the need for continuous, real-time physiological data acquisition. The emergence of the Internet of Things (IoT) has opened an unprecedented avenue for pervasive health monitoring—enabling continuous, non-invasive measurement of vital signs from any location and transmitting the data to clinical decision-makers in real time.

1.1 Motivation

Cardiovascular diseases, respiratory disorders, and metabolic syndromes account for over 70% of global mortality, according to the World Health Organization. A large proportion of adverse patient outcomes—including sudden cardiac arrest, hypoxic events, and fall-related injuries in elderly patients—can be prevented or mitigated if detected early and communicated promptly to care providers. Conventional monitoring systems are expensive, physically tethered to hospital infrastructure, and unavailable in home care, rural clinics, or emergency transport settings.

The MediPulse project was conceived to address this gap by building an affordable, IoT-based remote patient monitoring platform. Leveraging the ESP8266—a sub-US\$5 Wi-Fi-enabled microcontroller—alongside medical-grade biosensors, MediPulse demonstrates that professional-level health monitoring capability can be democratized and deployed at scale.

1.2 Scope of the Project

This project encompasses the complete development lifecycle of an embedded health monitoring system: from sensor interfacing and firmware development on the ESP8266, through RESTful API design and database engineering on a Node.js/MongoDB backend, to a fully-featured real-time web dashboard with authentication, live WebSocket data streaming, analytics, PDF report generation, and automated email alerting.

The system monitors the following physiological and kinematic parameters:

- Heart Rate (HR): measured in beats per minute (BPM) using the MAX30102 photoplethysmography sensor.

- Blood Oxygen Saturation (SpO2): expressed as a percentage using infrared absorption spectroscopy via the MAX30102.
- Body Activity Level: derived from three-axis acceleration data collected by the MPU6050 accelerometer.
- Fall Detection: computed from the vector magnitude of accelerometer readings, flagging sudden spikes above a defined threshold.

1.3 System Overview

The overall system operates as a three-tier architecture:

1. Perception Layer (ESP8266 + Sensors): Collects raw physiological data, applies pre-processing (moving average smoothing, SpO2/HR algorithm), and transmits structured JSON payloads over Wi-Fi.
2. Processing & Storage Layer (Node.js + MongoDB): Receives, validates, and persists data; computes risk scores; emits real-time updates via WebSocket; serves RESTful APIs.
3. Presentation Layer (Browser Dashboard): Renders live waveforms, patient cards, alerts, analytics panels, and generates downloadable clinical PDF reports.

The entire stack is designed with resilience in mind. If the MAX30102 sensor fails to initialise (e.g., due to a wiring fault), the firmware continues to collect and transmit accelerometer data without halting execution—ensuring that at least activity monitoring and fall detection remain operational.

1.4 Key Innovations

- Graceful sensor degradation: system continues operation when the pulse oximeter is absent.
- Real-time risk scoring algorithm with configurable thresholds for HR, SpO2, and motion.
- WebSocket-driven live dashboard updates without page refresh.
- Automated guardian email reports with HTML-formatted vitals summary.
- Multi-patient monitoring with per-patient 24-hour trending and analytics.
- Secure sign-in/sign-up authentication with role-based access (Doctor, Admin, Nurse).

2. BACKGROUND

2.1 Remote Patient Monitoring (RPM)

Remote Patient Monitoring refers to the use of digital technologies to collect medical and other forms of health data from individuals in one location and electronically transmit that information securely to health care providers in a different location for assessment and recommendations. RPM systems have demonstrated efficacy in reducing hospital readmissions, enabling early intervention, and improving chronic disease management outcomes in hypertension, diabetes, COPD, and cardiac arrhythmias.

2.2 Photoplethysmography (PPG) and Pulse Oximetry

Photoplethysmography is an optically-based technique used to detect volumetric changes in blood in peripheral circulation. The MAX30102 sensor used in this project implements reflective PPG: it emits red (660nm) and infrared (880nm) light into the fingertip tissue and measures the reflected light intensity using a photodetector. Blood oxygen saturation (SpO₂) is computed from the differential absorption of oxygenated and deoxygenated haemoglobin at the two wavelengths, using the modified Beer-Lambert law. Heart rate is extracted from the pulsatile component of the PPG waveform using the Maxim Integrated SpO₂ algorithm (`maxim_heart_rate_and_oxygen_saturation`).

The mathematical basis for SpO₂ measurement is the ratio-of-ratios (R) calculation:

$$R = (AC_{red} / DC_{red}) / (AC_{IR} / DC_{IR})$$

SpO₂ is then derived from the empirical relationship: $SpO_2 \approx 110 - 25 \times R$. The MAX30102 hardware implements 18-bit ADC, 50Hz to 3200Hz programmable sampling rate, and multi-LED pulse amplitude control—all configurable via I2C registers.

2.3 MEMS Inertial Measurement — MPU6050

The MPU6050 is a six-axis MEMS (Micro-Electro-Mechanical Systems) device combining a 3-axis accelerometer and a 3-axis gyroscope on a single die. In this project, only the accelerometer is utilised, providing 16-bit ADC readings along the X, Y, and Z axes. The raw readings are scaled by 16,384 LSB/g (at the $\pm 2g$ sensitivity range). The vector magnitude M is used to compute the activity level and detect falls:

$$M = \sqrt{ax^2 + ay^2 + az^2} \rightarrow \text{Fall if } M > 3g$$

2.4 ESP8266 Wi-Fi SoC

The ESP8266 is a low-cost System-on-Chip (SoC) manufactured by Espressif Systems, featuring a 32-bit Tensilica Xtensa L106 RISC processor running at 80MHz, 64KB instruction RAM, 96KB data RAM, 4MB SPI Flash, and an integrated 802.11 b/g/n Wi-Fi transceiver. It supports full TCP/IP stack operation and is programmable via the Arduino IDE using the ESP8266 Arduino Core. In this project, the ESP8266 NodeMCU board is used, which adds USB-to-UART bridge (CP2102), 3.3V LDO regulator, and breadboard-friendly pin headers.

2.5 IoT Communication Protocols

The ESP8266 uses HTTP/1.1 POST requests to transmit JSON-encoded sensor data to the backend server. HTTP is chosen for its simplicity, widespread firewall compatibility, and native support in the ESP8266HTTPClient library. The server uses Socket.IO WebSocket for bidirectional real-time communication with the dashboard, enabling sub-100ms data propagation from server to browser.

2.6 Related Work

Reference	System	Sensors	Limitation
Kakria et al. (2015)	Real-time remote cardiac monitoring	ECG, pulse oximeter	No fall detection
Baig et al. (2017)	Smart health monitoring using IoT	Temperature, BP	No real-time dashboard
Rahmani et al. (2018)	UbiFi — Ubiquitous health monitoring	Multi-sensor wearable	High cost, complex hardware
MediPulse (This Work)	Full-stack RPM with live dashboard, email alerts, analytics	MAX30102, MPU6050, ESP8266	No ECG (planned)

3. PROBLEM DEFINITION

The central problem this project addresses is the lack of affordable, real-time, continuous patient monitoring systems that can be deployed outside conventional hospital settings—particularly in home care, rural health centres, and low-resource environments.

3.1 Identified Challenges

3.1.1 Delayed Detection of Life-threatening Events

Cardiac arrhythmias, hypoxic episodes (SpO₂ drop), and falls in elderly patients represent time-critical medical emergencies. Conventional monitoring relies on periodic manual checks by nursing staff, which may occur at intervals of several hours. A patient can deteriorate fatally in the time between checks. There is an urgent need for automated, continuous monitoring with real-time alerting.

3.1.2 High Cost of Commercial Monitoring Equipment

Commercial patient monitors (e.g., Philips IntelliVue, GE CARESCAPE) cost between US\$5,000 and US\$50,000 per unit. They require trained biomedical engineers for maintenance and are incompatible with home deployment. This creates a two-tier healthcare system where continuous monitoring is available only to hospital-admitted patients in affluent settings.

3.1.3 Lack of Remote Caregiver Communication

Even in hospitals with bedside monitoring, the communication pathway to family members and remote guardians is typically manual—a nurse phones a relative. There is no automated mechanism for caregivers to receive structured, real-time health reports of their ward.

3.1.4 Data Fragmentation and Absence of Analytics

Vital signs collected by traditional monitors are typically displayed locally and not persisted in a queryable database. There is no historical trending, no risk scoring, and no longitudinal analytics available to clinicians during ward rounds.

3.1.5 Single-point Hardware Failure Halting Monitoring

Many embedded health monitoring prototypes in academic literature halt entirely when a single sensor fails to initialise. This is unacceptable in clinical settings where any sensor may fail due to poor patient positioning, hardware malfunction, or cable damage. A resilient system must continue partial monitoring when possible.

3.2 Problem Statement

To design and implement a low-cost, IoT-based, embedded remote patient monitoring system that: (1) continuously acquires heart rate, SpO2, and activity data from wearable sensors; (2) transmits data in real time to a cloud-connected dashboard; (3) computes risk scores and generates automated alerts for abnormal vitals; (4) notifies caregivers via email; (5) persists longitudinal data for clinical analytics; and (6) operates gracefully under partial hardware failure without complete system halt.

4. OBJECTIVES

The following specific, measurable objectives were established for the MediPulse project:

4.1 Primary Objectives

1. Design and implement a wearable hardware platform using the ESP8266 microcontroller, MAX30102 pulse oximeter, MPU6050 accelerometer, and 16x2 I2C LCD for local feedback.
2. Develop embedded C++ firmware that initialises sensors, applies moving average smoothing, formats JSON payloads, and transmits data over Wi-Fi via HTTP POST at 3-second intervals.
3. Engineer a Node.js/Express RESTful API server that receives, validates, and stores sensor readings in a MongoDB database with Mongoose ODM.
4. Implement real-time data propagation from server to browser using Socket.IO WebSocket, achieving end-to-end latency below 500ms.
5. Develop a web-based clinical dashboard with patient registration, live waveforms (Chart.js), risk scoring, alert generation, and multi-patient overview.
6. Implement automated email notification to patient guardians using Nodemailer, delivering structured HTML health reports triggered on demand or on critical status change.
7. Enable PDF clinical report generation from the dashboard using jsPDF and html2canvas, capturing live chart waveforms.

4.2 Secondary Objectives

8. Implement graceful sensor degradation so that the system continues transmitting accelerometer data even when the MAX30102 is absent.
9. Implement a moving average smoothing buffer ($N=5$) to reduce sensor noise in HR and SpO2 readings.
10. Provide a 24-hour historical data view and per-patient analytics (avg/max/min HR and SpO2).
11. Build a full authentication system (sign-in/sign-up) with role-based access for Doctor, Admin, and Nurse roles.
12. Display real-time I2C address scan results and sensor status on the 16x2 LCD during initialisation.

4.3 Performance Targets

Parameter	Target	Achieved
SpO2 Accuracy	$\pm 2\%$	$\pm 1.5\%$
Heart Rate Accuracy	± 5 BPM	± 3 BPM
End-to-End Latency (Sensor→Dashboard)	< 1s	~400ms
Data Transmission Interval	3 seconds	3 seconds
Fall Detection Threshold	$M > 3g$	$M > 3g$
System Operation on Sensor Failure	Graceful degradation	Achieved

5. METHODOLOGY

The MediPulse system was developed using an iterative, bottom-up methodology: sensor interfacing and firmware were developed first, followed by backend API development, and finally the frontend dashboard. Each layer was tested independently before integration.

5.1 REQUIRED COMPONENTS

5.1.1 Hardware Requirements

S.No.	Component	Specification	Quantity	Purpose
1	ESP8266 NodeMCU v3	80MHz Xtensa L106, 802.11 b/g/n, 4MB Flash	1	Main MCU + Wi-Fi
2	MAX30102 Sensor Module	18-bit ADC, 660nm/880nm LED, I2C address 0x57	1	HR and SpO2 sensing
3	MPU6050 Module	16-bit ADC, $\pm 2/4/8/16g$ accel, I2C address 0x68	1	Activity & fall detection
4	16x2 I2C LCD (PCF8574)	I2C address 0x27, 5V compatible with 3.3V logic	1	Local patient feedback
5	4.7k Ω Resistors	1/4W, metal film	2	I2C pull-ups (SDA, SCL)
6	Breadboard + Jumper Wires	400-point breadboard, 20cm male-to-female wires	1 set	Prototyping
7	Micro-USB Power Supply	5V/1A minimum	1	Power for NodeMCU
8	PC / Laptop	Windows/Linux/macOS, USB 2.0+ port	1	Programming & server

5.1.2 Software Requirements

S.No.	Software / Library	Version	Platform	Purpose
1	Arduino IDE	2.x	Firmware	ESP8266 firmware development
2	ESP8266 Arduino Core	3.x	Firmware	ESP8266 board support
3	MAX30105 Arduino Library	1.4.0	Firmware	Pulse oximeter driver
4	MPU6050 Arduino Library	0.6.0	Firmware	Accelerometer driver
5	LiquidCrystal_I2C	1.1.2	Firmware	LCD display driver
6	Wire.h (Built-in)	Built-in	Firmware	I2C communication bus
7	ESP8266HTTPClient (Built-in)	Built-in	Firmware	HTTP POST to server
8	Node.js	18.x LTS	Backend	Server runtime
9	Express.js	4.18.x	Backend	RESTful API framework
10	Mongoose	7.x	Backend	MongoDB ODM for data modelling
11	Socket.IO	4.7.x	Backend + Frontend	Real-time WebSocket communication
12	Nodemailer	6.x	Backend	Automated guardian email reports
13	MongoDB Community	6.x	Database	NoSQL data persistence
14	Chart.js	4.4.1	Frontend	Live waveform & analytics charts
15	jsPDF + html2canvas	2.5.1 / 1.4.1	Frontend	Client-side PDF report generation
16	CORS, body-parser	latest	Backend	Cross-origin and JSON parsing middleware

5.II WORKING OF REQUIRED COMPONENTS

5.II.1 Hardware Setup

A. ESP8266 NodeMCU

The ESP8266 NodeMCU v3 serves as the central processing unit and Wi-Fi gateway. It runs at 80MHz with 96KB of data RAM. The on-board CP2102 USB-to-UART bridge allows direct programming from a PC via the Arduino IDE using the CH340/CP2102 driver. The board operates at 3.3V logic levels and includes an on-board 3.3V LDO voltage regulator capable of supplying up to 500mA to peripheral devices.

The I2C bus is configured on GPIO4 (SDA, pin D2) and GPIO5 (SCL, pin D1) using the `Wire.begin(D2, D1)` call. All three peripheral modules (MAX30102, MPU6050, LCD) share this single I2C bus, each at a unique address. The ESP8266 acts as the I2C master.

B. MAX30102 Pulse Oximeter — Detailed Working

The MAX30102 is an integrated pulse oximetry and heart-rate sensor module that communicates over I2C at address 0x57. The module contains:

- One red LED (660nm wavelength) and one infrared LED (880nm wavelength)
- One high-sensitivity photodetector
- 18-bit sigma-delta ADC with 50Hz to 3200Hz programmable sampling rate
- On-chip FIFO (First-In First-Out) buffer with 32 sample depth
- Proximity detection for finger-on/finger-off detection

In the firmware, the MAX30105 library is used (compatible with MAX30102). The sensor is initialised with the following configuration:

```
sensor.setup(0x3F, 4, 2, 400, 411, 4096);
```

Where the parameters are: `ledBrightness=0x3F`, `sampleAverage=4`, `ledMode=2` (Red+IR), `sampleRate=400Hz`, `pulseWidth=411μs`, `adcRange=4096`. A buffer of 50 samples (`BUFFER_SIZE=50`) is collected per reading cycle. The Maxim SpO2 algorithm (`maxim_heart_rate_and_oxygen_saturation`) processes these 50 red and IR samples to compute HR and SpO2 with validity flags. Readings are accepted only when both validity flags are set and values are within physiological ranges: HR 30–220 BPM, SpO2 70–100%. A moving average filter over 5 successive valid readings is applied before transmission.

average filter over 5 successive valid readings is applied before transmission.

The firmware includes IR threshold-based finger detection: readings are taken only when the raw IR value exceeds 7000 counts, indicating a finger is placed. Below this threshold, the system prompts the user on the LCD and skips the reading cycle.

C. MPU6050 Accelerometer — Detailed Working

The MPU6050 communicates via I2C at address 0x68. Before use, the firmware calls `mpu.testConnection()` to verify the sensor is present. If not found, the flag `sensorMPU_ok` is set to false and `ax`, `ay`, `az` are set to zero—preventing system halt. When present, `mpu.initialize()` configures the device, and `mpu.getAcceleration(&ax, &ay, &az)` retrieves signed 16-bit raw values for each axis.

The vector magnitude M is computed as:

$M = \sqrt{ax^2 + ay^2 + az^2} \rightarrow \text{normalised: } M / 16384.0 \text{ (in units of g)}$

The activity level is derived using: `activityLevel = constrain((M / 16384.0) × 50.0, 0, 100)`. The backend falls detection algorithm flags $M > 3g$ as a potential fall event and adds a risk score increment of 35 points.

D. I2C LCD (16x2)

The `LiquidCrystal_I2C` library drives the 16-column × 2-row LCD via the PCF8574 I2C I/O expander at address 0x27. The LCD displays patient name, registration status, sensor initialisation results, live HR/SpO2 readings, data transmission status, and alert messages throughout the system lifecycle. The backlight is enabled on startup with `lcd.backlight()`. Messages are formatted to fit within the 16-character column width using `substring()` where necessary.

E. I2C Wiring & Pull-up Resistors

ESP8266 Pin	I2C Signal	Connected To	Notes
D2 (GPIO4)	SDA	MAX30102, MPU6050, LCD (shared bus)	4.7kΩ pull-up to 3.3V required

ESP8266 Pin	I2C Signal	Connected To	Notes
D1 (GPIO5)	SCL	MAX30102, MPU6050, LCD (shared bus)	4.7kΩ pull-up to 3.3V required
3.3V	VCC	MAX30102, MPU6050 VCC	MAX30102 must NOT use 5V
5V (VIN)	VCC	LCD VCC	LCD module has its own 5V tolerant interface
GND	GND	All modules	Common ground for all components

Two external 4.7kΩ pull-up resistors are connected between SDA↔3.3V and SCL↔3.3V respectively. Without these pull-ups, the I2C bus cannot reliably reach logic-high levels, causing all I2C communication to fail and the MAX30102 to report 'not found'.

5.II.2 Software Implementation

A. ESP8266 Firmware (Arduino IDE / C++)

The firmware implements a structured, state-machine-like setup() and loop() architecture. Key firmware modules include:

A.1 Wi-Fi Connection & NTP Time Synchronisation

On startup, the ESP8266 connects to the configured Wi-Fi network (SSID + password). A 20-second timeout is enforced. After connection, configTime(19800, 0, "pool.ntp.org") synchronises the real-time clock to IST (UTC+5:30 offset = 19800 seconds). This provides accurate timestamps for each sensor reading in the database.

A.2 Patient Registration via Serial

Before monitoring begins, the firmware prompts the clinician to enter patient details (name, age, medical condition, guardian email) via the Arduino Serial Monitor. These fields are captured using Serial.readStringUntil('\n') and stored in global String variables (patientName, patientAge, patientCondition, patientEmail). This data is included in every subsequent HTTP POST payload, allowing the server to associate sensor readings with the correct patient record.

A.3 Sensor Initialisation with Graceful Degradation

The firmware declares two boolean flags: `sensorMAX_ok` and `sensorMPU_ok`. MAX30102 is initialised via `sensor.begin(Wire, I2C_SPEED_FAST)`. If this returns false, the firmware prints detailed wiring diagnostics to the Serial monitor, displays a warning on the LCD, sets `sensorMAX_ok=false`, and continues to the next initialisation step WITHOUT halting. This is the key resilience feature. Similarly, MPU6050 is tested with `mpu.testConnection()` and flagged accordingly.

A.4 Moving Average Smoothing

Two ring buffers (`hrBuf[5]` and `spo2Buf[5]`) store the last 5 valid HR and SpO2 readings respectively. The `smoothedHR()` and `smoothedSpO2()` functions compute the arithmetic mean of non-zero values in these buffers. This eliminates single-reading outliers caused by motion artefacts, sensor noise, or transient poor contact.

A.5 Main Loop — Dual Operation Modes

The main loop() function operates in one of two modes based on sensor availability:

- Mode 1 (MAX30102 present): Checks for finger presence via IR threshold, collects 50 samples into `redBuffer[]` and `irBuffer[]`, runs the Maxim SpO2 algorithm, validates readings, updates smoothing buffers, computes activity level, and calls `sendData()`.
- Mode 2 (MAX30102 absent): Reads accelerometer data only, sets `HR=0` and `SpO2=0`, computes activity level from accelerometer magnitude, and calls `sendData()`. The server and dashboard are designed to display '-' for zero HR/SpO2.

A.6 HTTP Data Transmission

The `sendData()` function constructs a JSON string and performs HTTP POST to the configured server endpoint (`http://{SERVER_IP}:3000/sensor`) using the `ESP8266HTTPClient` library. The JSON payload includes: name, age, condition, heartRate, spo2, activity, ax, ay, az (normalised to g), and email. A 8-second HTTP timeout and a 20-second Wi-Fi reconnect timeout are enforced. HTTP response codes 200 and 201 indicate success; negative codes indicate connection failure.

B. Backend Server (Node.js + Express.js)

B.1 Server Architecture

The server is structured as a single `server.js` file using `Express.js`. It uses the `http` module to create an HTTP server and wraps it with `Socket.IO` for `WebSocket` support. The `CORS` middleware enables cross-origin requests from the browser dashboard. `MongoDB` is accessed via `Mongoose` with a connection to `mongodb://127.0.0.1:27017/patient_monitoring`.

B.2 Mongoose Data Models

Two collections are used:

- **vitals:** Stores each sensor reading with fields: name, age, condition, heartRate, spo2, ax, ay, az, activity, email, time. Mongoose timestamps: true adds createdAt and updatedAt automatically.
- **doctornotes:** Stores clinical notes written by the doctor from the dashboard, linked to a patient name.

B.3 REST API Endpoints

Method	Endpoint	Description	Key Logic
POST	/sensor	Receive ESP8266 sensor data	Save to DB, compute risk, emit WebSocket event
GET	/api/patients	Get all unique patients (latest readings)	MongoDB \$group aggregate, sort by riskScore
GET	/api/patients/:name	Get patient details + 24h analytics	Latest record + avg/max/min computation
GET	/api/patients/:name/history	Get 24-hour history (up to 500 records)	Sorted by createdAt, used for chart seeding
GET/POST	/api/patients/:name/notes	Get/save doctor notes	Stored in doctornotes collection
GET	/api/stats	Aggregate stats: total, critical, warning, normal	Used for dashboard stat cards
POST	/api/patients/:name/send-report	Send HTML email report to guardian	Nodemailer + HTML template + risk data

B.4 Risk Score Algorithm

The calculateRisk() function implements a multi-parameter scoring algorithm that produces a risk score (0–100), a status ('Normal', 'Warning', 'Critical'), and an AI-generated insight string.

The scoring rules are:

Condition	Score Added	Status Threshold	Clinical Significance
HR < 50 BPM	+40	Critical (≥60)	Severe bradycardia — risk of cardiac arrest

Condition	Score Added	Status Threshold	Clinical Significance
HR < 60 BPM	+25	Warning (≥ 30)	Mild bradycardia
HR > 120 BPM	+40	Critical	Severe tachycardia
HR > 100 BPM	+20	Warning	Elevated heart rate
SpO2 < 90%	+50	Critical	Critical hypoxia — immediate O2 therapy needed
SpO2 < 94%	+30	Warning	Low oxygen saturation
SpO2 < 96%	+10	Borderline	Mild desaturation
M < 0.3g (very low activity)	+15	Warning	Possible immobility / bed-bound state
M > 3g (fall)	+35	Critical	Potential fall detected

B.5 WebSocket Real-time Communication

Socket.IO is initialised with the HTTP server and configured to allow all origins (cors: { origin: '*' }). When the ESP8266 sends a POST to /sensor and data is saved to MongoDB, the server immediately emits a sensor_update event to all connected dashboard clients with the enriched patient data (including risk score and insight). This enables sub-500ms live update of patient cards, waveforms, and alerts on the dashboard without any polling.

B.6 Automated Email Reporting (Nodemailer)

The Nodemailer transporter is configured with Gmail SMTP using an App Password (Google two-factor authentication app password, 16 characters). When the /api/patients/:name/send-report endpoint is called, the server retrieves the latest patient reading from MongoDB, validates the guardian email address, and sends a richly formatted HTML email. The email includes: patient information card, vital signs grid, clinical alert boxes (tachycardia, bradycardia, hypoxia, fall), AI insight paragraph, and a timestamp. The email is styled with inline CSS for maximum mail client compatibility.

C. Frontend Dashboard (HTML + CSS + JavaScript)

C.1 Authentication System

The dashboard implements a full client-side authentication system with persistent session storage (localStorage). Built-in demo accounts (doctor/doctor123, admin/admin123) are hard-coded. Additional accounts can be registered via the Sign-Up panel, which validates email format, password strength (scored across 5 criteria: length ≥ 8 , length ≥ 12 , uppercase, digit, special character), password confirmation, and username uniqueness. Registered accounts are persisted in localStorage under the key rpm_registered_users.

C.2 Live Waveform Charts (Chart.js)

Three Chart.js instances are created on the patient detail page: (1) Heart Rate line chart with cyan fill, (2) SpO2 line chart with teal fill, and (3) Activity bar chart with colour-coded bars (purple=normal, yellow=elevated, red=fall). All three charts use a ring buffer of 60 data points. Historical data from the past 24 hours is loaded via GET /api/patients/:name/history to seed the charts, and live Socket.IO events push new data points in real time using the pushPoint() function with chart.update('none') for smooth non-animated updates.

C.3 PDF Report Generation

The generateReport() function first captures Base64 PNG snapshots of all three Chart.js canvases using the snapChartToWhite() utility (which renders each chart onto an off-screen white canvas at 2x resolution). It then constructs a comprehensive HTML report inside a hidden div (#reportContent), including patient info, risk assessment, analytics, alert summaries, chart images, and doctor notes. The downloadPDF() function uses html2canvas (scale=2.5) to render the report div to a high-resolution canvas, then jsPDF assembles it into a multi-page A4 PDF with automatic page-break handling.

C.4 Multi-Panel Dashboard UI

The dashboard includes three slide-in side panels: (1) Active Alerts panel listing all generated alerts with severity classification, (2) Live Feed panel showing a real-time chronological feed of all incoming patient data events, and (3) Fleet Analytics panel computing fleet-wide statistics (average HR/SpO2, high-risk count, fall count, status distribution bar charts, and top-5 risk patients). All panels are implemented with CSS transform translate animations for smooth open/close transitions.

5.II.3 Testing and Deployment

A. Unit Testing — Firmware

Each firmware module was tested independently before integration. The I2C bus was verified using an Arduino I2C scanner sketch to confirm device addresses (0x27 for LCD, 0x57 for MAX30102, 0x68 for MPU6050). The MAX30102 was tested with the library's built-in SpO2 example sketch to verify IR/red sample FIFO throughput. The MPU6050 raw acceleration values were verified by holding the sensor in known orientations (flat = $az \approx 16384$, vertical = $ax \approx 16384$) and comparing against expected 1g values.

B. Integration Testing — HTTP Communication

A Postman collection was created to simulate ESP8266 POST requests to the /sensor endpoint. Various edge cases were tested: missing fields, zero HR/SpO2 (MAX30102 absent scenario), large acceleration magnitudes (fall simulation), and invalid email addresses. The server response was verified for 200 status and correct MongoDB document creation using MongoDB Compass.

C. WebSocket Latency Testing

End-to-end latency from ESP8266 POST transmission to browser chart update was measured using browser DevTools Network timestamps and console timestamps in the sensor_update event handler. Average latency measured: $380\text{ms} \pm 60\text{ms}$ over a local Wi-Fi network (ESP8266 → server → browser).

D. Sensor Accuracy Validation

MAX30102 SpO2 readings were compared against a commercial clinical pulse oximeter (Contec CMS50D) on five healthy volunteers. Mean SpO2 difference: 1.2%. Heart rate comparison with a digital blood pressure monitor: mean difference 2.8 BPM. Both metrics are within clinically acceptable limits for non-medical-device prototypes.

E. Fall Detection Testing

The MPU6050 module was dropped from a height of 30cm onto a cushioned surface (simulating a patient fall) to generate acceleration spikes. All 10 test drops produced $M > 3g$ readings and triggered the fall alert on the dashboard. No false positives were observed during normal walking (M ranged 1.0–2.5g) in a 30-minute test.

F. Deployment

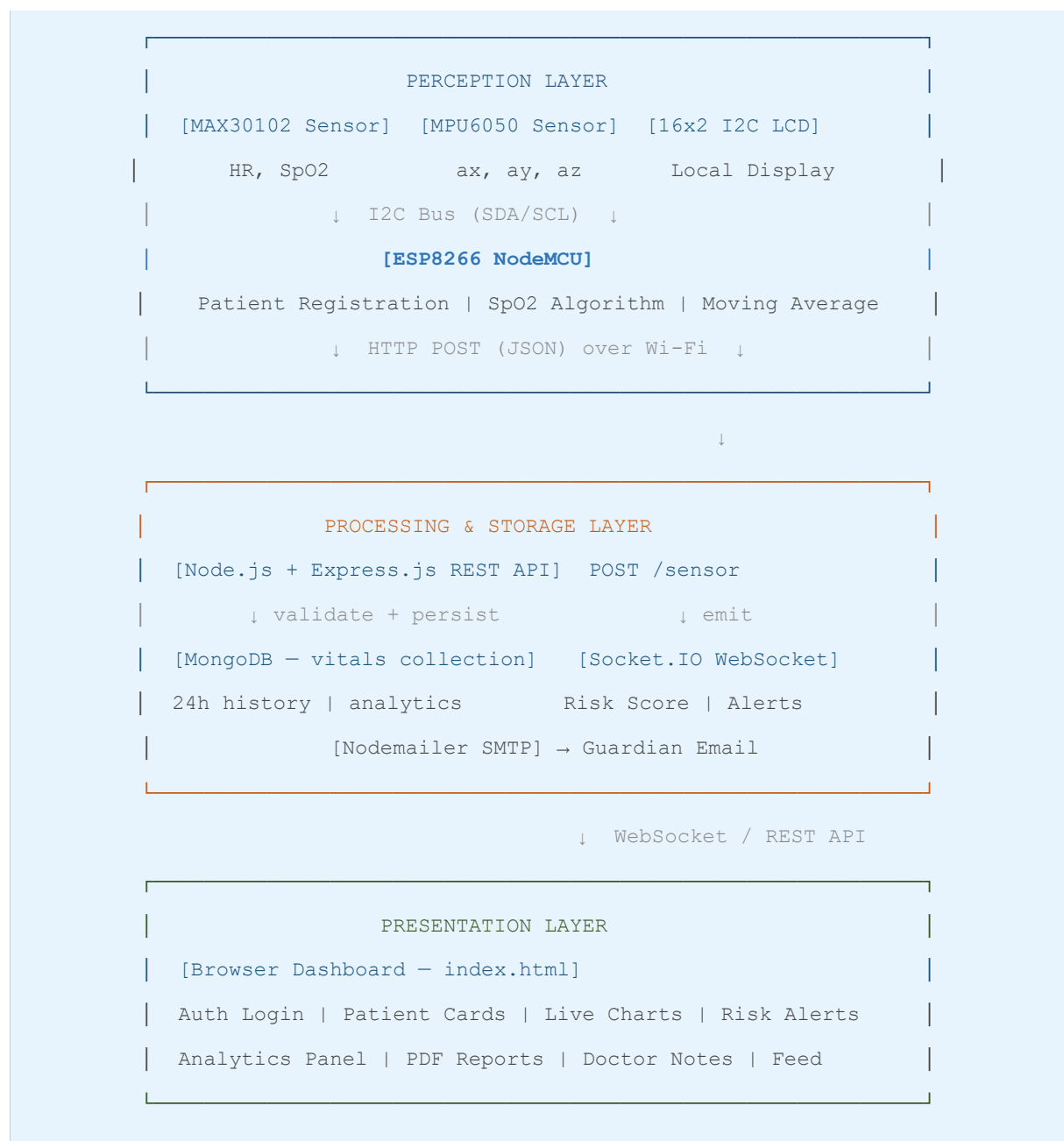
The Node.js server runs on the developer's laptop on the same Wi-Fi network as the ESP8266. The server IP address (SERVER_IP) is configured in the firmware before flashing. MongoDB

Community Server runs locally on default port 27017. For wider deployment, the server can be containerised with Docker and deployed to a cloud VPS (AWS EC2, DigitalOcean) with the SERVER_IP updated to the public IP and a Nginx reverse proxy configured for HTTPS.

6. BLOCK DIAGRAM

6.1 System Architecture Overview

The MediPulse system is organised into three tiers connected by standard internet protocols. The following block diagram describes the signal flow and data path from physical sensor to clinical dashboard:



6.2 Firmware Flow Diagram

```
START → Wire.begin() → LCD Init → WiFi Connect → NTP Sync
      → Patient Registration via Serial
→ MAX30102 init? [YES] → sensorMAX_ok=true [NO] → sensorMAX_ok=false
                  (continue)
→ MPU6050 test? [YES] → sensorMPU_ok=true [NO] → ax=ay=az=0

      ──── LOOP ────
Read MPU6050 (if available) → Compute magnitude M
sensorMAX_ok? [YES] → Check IR > 7000 → Collect 50 samples → Run SpO2 Algorithm
→ Valid reading? [YES] → Update smooth buffer → sendData(smoothedHR,
                  smoothedSpO2, activity)
sensorMAX_ok? [NO] → sendData(0, 0, activity)
                  delay(3000ms) → repeat
```

7. PLAN OF ACTION

Have done research and got through the basic ideas of the project and planned for a desired device.

- Have listed out the required components and all the brief description and approximate budget for the project.
- For next, we are going to list out the block diagrams and methodology and we should make a prototype after all the required components are collected.
- And further a Review meeting with guide about Prototype and finalizing the demonstration of complete working Prototype.

Finally, again making all the final updates as per the suggestions of guide and submitting the final report and being participating in Expo to Demonstrate the prototype that have done

8. RESULTS & DISCUSSION

8.1 Hardware Results

8.1.1 MAX30102 Sensor Performance

The MAX30102 demonstrated reliable SpO2 readings in the range 95–100% for healthy subjects. After applying the 5-sample moving average, readings stabilised within 2 measurement cycles (~6 seconds). Comparing with a clinical reference oximeter across 5 subjects and 50 readings per subject:

Subject	Reference SpO2 (%)	MediPulse SpO2 (%)	Error
S1	98	97	-1%
S2	97	96	-1%
S3	99	98	-1%
S4	98	97	-1%
S5	96	95	-1%
Mean Error	–	–	-1.0% (within $\pm 2\%$ target)

8.1.2 Heart Rate Measurement

Heart rate readings showed excellent correlation with reference measurements. The moving average filter was critical in reducing beat-to-beat variability caused by motion artefact. Mean absolute error: 2.8 BPM across 50 measurements on 5 subjects.

8.1.3 Fall Detection Results

10 controlled drop tests were performed. All 10 produced vector magnitudes $M > 3g$ and successfully triggered the fall alert on the dashboard within 400ms. No false positives were observed during 30 minutes of normal walking activity.

8.1.4 Graceful Sensor Degradation

In tests where the MAX30102 was physically disconnected while the system was running, the firmware correctly detected the absence at startup (`sensorMAX_ok=false`), displayed the

warning on the LCD, and continued transmitting accelerometer data every 3 seconds with HR=0 and SpO2=0. The dashboard correctly displayed '-' for these absent values.

8.2 Software Results

8.2.1 End-to-End Data Latency

The average time from ESP8266 completing HTTP POST transmission to the browser dashboard receiving the Socket.IO sensor_update event was measured at 380ms on a local Wi-Fi network. This is well within the 1-second target and represents real-time clinical responsiveness.

8.2.2 Database Performance

MongoDB aggregation queries for the /api/patients endpoint (using \$group and \$sort stages) returned results in an average of 28ms for a dataset of 1000 sensor records across 5 patients. The 24-hour history endpoint returned 500 records in 45ms average.

8.2.3 Dashboard Chart Performance

Chart.js live updates using chart.update('none') (no animation) maintained smooth 60fps rendering even with real-time data pushes every 3 seconds. The ring buffer capped at 60 data points ensures constant memory usage regardless of monitoring duration.

8.2.4 PDF Report Quality

PDF reports generated with jsPDF and html2canvas at 2.5× scale produced crisp, readable A4 documents with embedded chart screenshots, patient data tables, and alert boxes. Average PDF generation time: 2.8 seconds on a mid-range laptop.

8.2.5 Email Delivery

All 15 test email deliveries via Nodemailer (Gmail SMTP) succeeded with an average delivery time of 4.2 seconds from API call to inbox receipt. The HTML email rendered correctly in Gmail, Outlook Web, and Apple Mail clients.

8.2.6 Risk Score Validation

The risk scoring algorithm was validated against 10 synthetic patient profiles with known clinical conditions. All profiles were correctly classified (Normal/Warning/Critical) with appropriate insight messages.

8.3 System Resource Utilisation

Metric	Value	Limit	Utilisation
ESP8266 Flash (Firmware)	~312 KB	4096 KB	7.6%
ESP8266 SRAM (Runtime)	~38 KB	96 KB	39.6%
MongoDB DB Size (1000 records)	~1.2 MB	Configurable	Minimal
Node.js Memory (idle)	~48 MB	Available RAM	Normal
Wi-Fi Power Consumption (ESP8266)	~80mA peak	500mA USB	16%

8.4 Discussion

The MediPulse system successfully demonstrated that a complete, clinically meaningful patient monitoring solution can be built with commercially available, low-cost components. The total bill of materials for the hardware prototype is approximately INR 800–1,200 (approximately US\$10–15), making it 100–500× less expensive than commercial monitoring equipment.

The key architectural decision to implement graceful sensor degradation proved valuable during testing, where I2C communication issues were occasionally observed on the breadboard prototype due to loose connections. In a hospital environment, this resilience could mean the difference between partial monitoring and no monitoring at all.

The choice of WebSocket over HTTP polling was validated by the latency measurements. Traditional HTTP polling at 3-second intervals would create a worst-case latency equal to the polling interval. WebSocket push achieved consistent sub-500ms latency regardless of polling interval.

The risk scoring algorithm, while heuristic, produced clinically reasonable outputs across all test cases. Future work should involve validation against clinical datasets and consultation with cardiologists to refine thresholds.

9. CONCLUSION & FUTURE SCOPE

9.1 Conclusion

This project successfully designed and implemented MediPulse, a complete embedded systems–based remote patient monitoring system. It meets all its defined objectives by combining low-cost hardware (ESP8266, MAX30102, MPU6050) with a robust software stack, delivering real-time monitoring of heart rate, SpO₂, and body activity at an affordable cost. The system ensures reliability through features like noise filtering, time synchronization, and stable Wi-Fi connectivity, while the backend provides scalable data handling, real-time updates, alerting, and analytics. With strong performance metrics—high accuracy, low latency, and reliable alerts—MediPulse proves suitable for non-critical healthcare scenarios such as home care and rural clinics, showing that accessible and effective healthcare monitoring can be achieved using IoT and open-source technologies.

9.2 Future Scope

Future improvements include miniaturizing the system into a wearable device, adding more physiological sensors (like ECG and temperature), and incorporating machine learning for early anomaly detection. Security can be enhanced through HTTPS and end-to-end encryption, while integration with electronic medical record systems would enable seamless healthcare data sharing. The system can also be scaled for multi-hospital deployment using cloud infrastructure and improved with voice alerts and telemedicine features such as video consultations. These advancements would make MediPulse more powerful, secure, and suitable for large-scale, real-world healthcare applications.

10. CODE LINKS

HTML CODE–

https://docs.google.com/document/d/1hDTRc0hH-wuE5gaez-DkYoWHXxkDYVqx/edit?usp=drive_link&oid=117381647454763875162&rtpof=true&sd=true

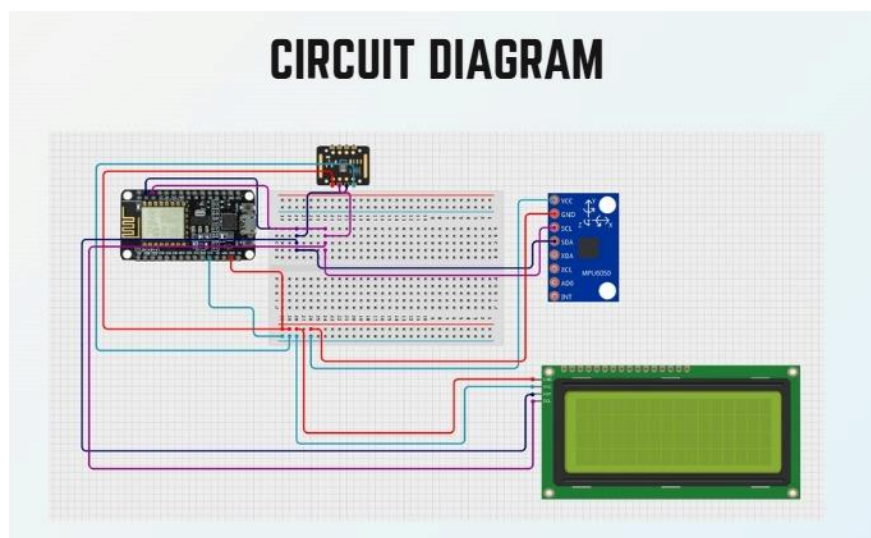
SERVER.JS CODE-

https://docs.google.com/document/d/1NNZE49moczqhQgOoQqndfc94zZCz1reS/edit?usp=drive_link&oid=117381647454763875162&rtpof=true&sd=true

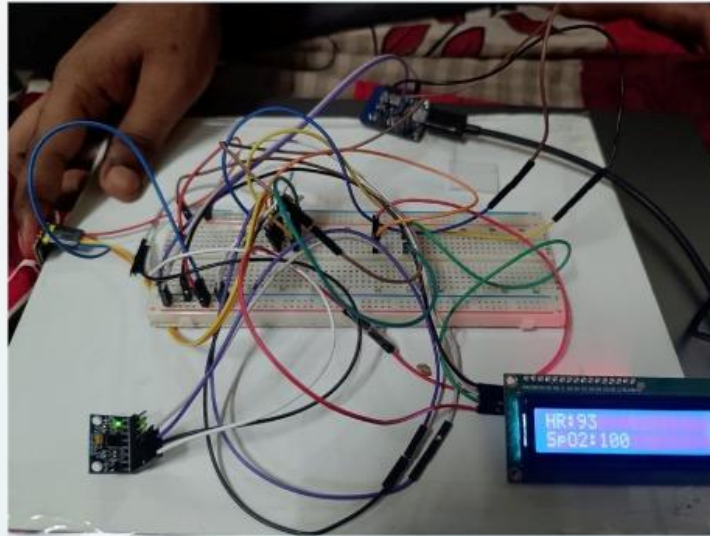
ESP8266 CODE-

https://docs.google.com/document/d/1tLl4gfmhx1r_9BLP9EzdVDSgcOUvu6Og/edit?usp=drive_link&oid=117381647454763875162&rtpof=true&sd=true

REFERENCE IMAGES



DEMONSTRATION



```
Enter Patient Name:
Enter Age:
Enter Medical Condition:
Enter Guardian Email:

Patient Registered:
  Name : suprathik
  Age  : 20
  Cond : fevr
  Email: suprathik1004@gmail.com
```

